

Tarea: investigación patrones de diseño

Los patrones de diseño son soluciones reutilizables a problemas comunes que aparecen durante el desarrollo de software. No son código exacto, sino formas probadas de organizar clases, objetos y responsabilidades para construir sistemas más mantenibles, flexibles y escalables.

1. Factory Method

El patrón Factory Method permite crear objetos sin depender directamente de una clase concreta. En lugar de usar new en todas partes, se delega la creación a un método especializado. Esto facilita cambiar implementaciones sin modificar el código principal.

Lo entiendo como una fábrica que decide qué objeto crear dependiendo de una necesidad específica.

Se usan cuando existen varios tipos de objetos relacionados cuando queremos desacoplar la creación del uso.

2. Abstract Builder

Este patrón permite crear familias completas de objetos relacionados sin especificar sus clases concretas.

Yo lo entiendo como una fábrica más avanzada, donde no solo se crea un objeto, sino varios objetos relacionados entre sí.

Se usa en interfaces gráficas y sistemas multiplataforma.

3. Builder

Builder permite construir objetos complejos paso a paso, separando la construcción de la representación final.

Lo veo como armar una computadora pieza por pieza y Lo usamos en objetos con muchos parámetros y configuraciones complejas.

4. Prototype Este patrón permite crear nuevos objetos copiando uno existente lo entiendo cómo sacar una fotocopia de un objeto ya configurado.

Lo usamos para crear objetos desde cero es costoso.

5. Singleton

Singleton garantiza que solo exista una instancia de una clase es como una oficina central que controla un recurso compartido.

Lo usamos en configuraciones y conexiones globales

6. Adapter

Adapter permite que clases incompatibles trabajen juntas es como un adaptador de corriente y se utiliza en librerías y apis antiguas.

7. Bridge

Bridge separa una abstracción de su implementación se entiende como separar el control remoto del televisor.

Y se usa en combinaciones posibles.

8. Composite

Composite permite tratar objetos individuales y grupos de objetos de la misma manera como carpetas y archivos en Windows.

Se utiliza en árboles, menús y jerarquías.

El proyecto consiste en una API administrativa para manejar clientes, compras, ventas y reportes de una bodega, incluyendo control de inventario, notificaciones por correo y mensajería con RabbitMQ. Analizando la arquitectura del sistema, identifiqué los siguientes patrones de diseño que pueden aportar flexibilidad, organización y escalabilidad.

1. **Singleton:** Para manejar la conexión a base de datos.
2. **Builder:** Para construir ventas y compras complejas.
3. **Factory Method.** Para generar documentos comerciales.
4. **Adapter.** Para integrar servicios externos.
5. **Bridge.** Para exportación de reportes.
6. **Composite.** Para paquetes de productos.

Estos patrones permiten que el sistema sea más organizado, escalable y fácil de mantener conforme el negocio crezca.